

PROJECT REPORT

Customer Support AI Agent

Design, Integration Challenges, and Production Readiness

Built on Amazon Bedrock AgentCore & Strands Agents

Prepared by

Christopher Masukume

September 2026

1. Architecture & Design Decisions

A key design decision in this project was to use Amazon Bedrock AgentCore as the central runtime and orchestration layer, while separating specialized capabilities into dedicated tools. Rather than embedding every capability directly into the agent, each function is delegated to a purpose-built service that the agent calls into as needed.

Component Responsibilities

AgentCore Gateway	Exposes order-tracking and refund-processing services to the agent as callable tools.
Bedrock Knowledge Base	Provides product and policy information for grounded, accurate responses.
AgentCore Memory	Retains customer context (name, preferences, prior interactions) across sessions.
AgentCore Browser	Retrieves live web information when a customer's question requires up-to-date, external data.

This modular approach makes the agent easier to extend: individual capabilities can be updated, replaced, or scaled independently without redesigning the application as a whole.

2. Integration Challenges

A major challenge was integrating the different asynchronous components, particularly the AgentCore Browser and Strands Agents. The browser tool uses Playwright and its own event loop, while the AgentCore runtime has its own execution environment. This mismatch surfaced as a runtime error — “Timeout should be used inside a task” — when testing the live browser functionality.

Troubleshooting Process

Resolving the issue required a structured investigation across several layers of the system:

- Examining AgentCore runtime logs to trace where in the request lifecycle the failure occurred.
- Reviewing dependency versions to rule out incompatibilities between the Strands Agents SDK and the browser tooling.
- Inspecting the browser tool implementation to understand its reliance on an active asyncio event loop.
- Verifying AgentCore Browser connectivity to confirm the managed browser session itself was reachable and provisioning correctly.

Separately, development was interrupted by expired temporary AWS credentials, which required refreshing the VocLabs session before testing could continue — a reminder that short-lived credential lifecycles need to be accounted for in any development workflow that involves iterative, hands-on testing.

3. Production Readiness Considerations

Moving this agent from a lab environment to production would require attention across four areas: security, reliability, observability, and scalability.

Security

- Strengthen authentication and authorization on the AgentCore Gateway rather than relying on an unauthenticated endpoint.
- Protect customer data stored in long-term memory through appropriate IAM permissions and data-governance controls.

Reliability & Observability

- Introduce production monitoring that captures agent failures, tool latency, memory operations, and downstream service errors.
- Establish alerting thresholds so degraded performance in any single tool (Gateway, Knowledge Base, Memory, or Browser) is caught before it affects customers.

Scalability & Delivery

- Introduce automated testing and deployment pipelines so that changes to the agent, tools, prompts, and dependencies are validated before reaching production.
- Plan capacity around the components most likely to bottleneck under load, particularly Gateway throughput and Browser session provisioning.

4. Written Reflection

Customer Support AI Agent — Design, Integration Challenges, and Production Readiness

One of the most important design decisions I made in this project was to use Amazon Bedrock AgentCore as the central runtime and orchestration layer, while exposing specialised capabilities through separate tools. In particular, I used AgentCore Gateway with MCPClient to connect the agent to order-tracking and refund services. This kept business operations separate from the agent logic and made the integration easier to test and extend. I also used AgentCore Memory for customer context and the AgentCore Browser for live web information.

A major challenge I encountered was integrating the Browser with the asynchronous AgentCore and Strands execution environment. Early browser testing produced the error “Timeout should be used inside a task”, and later the deployed runtime exceeded its 30-second initialization window. I investigated the runtime behaviour, tested the managed browser independently, adjusted the browser configuration, and identified the need for the appropriate IAM permissions to access the AWS-managed Browser resource. After the permission was added, the deployed agent successfully navigated to a live Udacity page and returned its page title. This showed me the importance of testing each integration independently before diagnosing the whole application.

For production, I would place particular emphasis on security, monitoring, and scalability. The Gateway should use stronger authentication and authorization rather than an unauthenticated endpoint, while customer memory should be protected with least-privilege IAM permissions and appropriate data-governance controls. I would also monitor agent failures, Gateway and tool latency, memory operations, and downstream service errors. Finally, automated testing and deployment pipelines would help validate changes before release, while capacity planning would be important for Gateway throughput and Browser session provisioning.